



المدرسة الوطنية للعلوم التطبيقية - بني ملال

ⵜⴰⵎⴰⵔⵜ ⵜⴰⵎⴰⵏⴰⵢⵜ ⵜⴰⵖⴰⵏⴰⵏⵜ ⵜⴰⵙⴰⵎⴰⵏⴰⵢⵜ - ⵎⴰⵎⴰⵍ ⵔⴰⵎⴰⵏⴰⵢⵜ

Ecole Nationale des Sciences Appliquées - Béni Mellal

Le rapport du TP°4 : Architecture Orientée Services (SOA)

Réalisé par :

Hafssa BOUCHBOUK

Encadré par :

Pr. BE ELBAGHAZAOU

Introduction

Ce TP porte sur l'architecture orientée services (SOA), un modèle de conception qui consiste à décomposer une application en plusieurs services indépendants et communicants. Dans ce contexte, on développe une application e-commerce simple basée sur des microservices avec Spring Boot. Chaque service possède sa propre responsabilité et communique avec les autres via des API REST, tout en étant enregistré et découvert grâce à un serveur Eureka. Cette approche permet de simuler une architecture moderne, modulaire et évolutive, proche des systèmes utilisés dans les environnements professionnels.

Objectifs du TP

Ce TP vise à :

- Comprendre les principes fondamentaux de l'architecture SOA
- Mettre en place et manipuler des microservices avec Spring Boot
- Implémenter la communication entre services via REST et OpenFeign
- Utiliser Spring Data JPA pour gérer les données
- Mettre en place un service de découverte avec Eureka
- Tester et valider l'interaction entre les différents services à l'aide de Postman

Étape 1 : Mise en place du Serveur de Découverte (Eureka)

1-Configuration

Cette étape consiste à configurer le serveur Eureka via le fichier **application.properties**. Le port du serveur est défini sur **8761**, et son nom d'application est **eureka-server**. Les options **register-with-eureka** et **fetch-registry** sont désactivées, car ce service joue uniquement le rôle de registre et ne s'enregistre pas lui-même ni ne récupère d'autres services. Enfin, l'URL de base d'Eureka est définie pour permettre aux autres microservices de s'y connecter. Cette configuration permet de mettre en place un service central de découverte des microservices dans l'architecture SOA.

```
eureka-server > src > main > resources > ⚙ application.properties
1  server.port=8761
2  spring.application.name=eureka-server
3  eureka.client.register-with-eureka=false
4  eureka.client.fetch-registry=false
5  eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
6  |
```

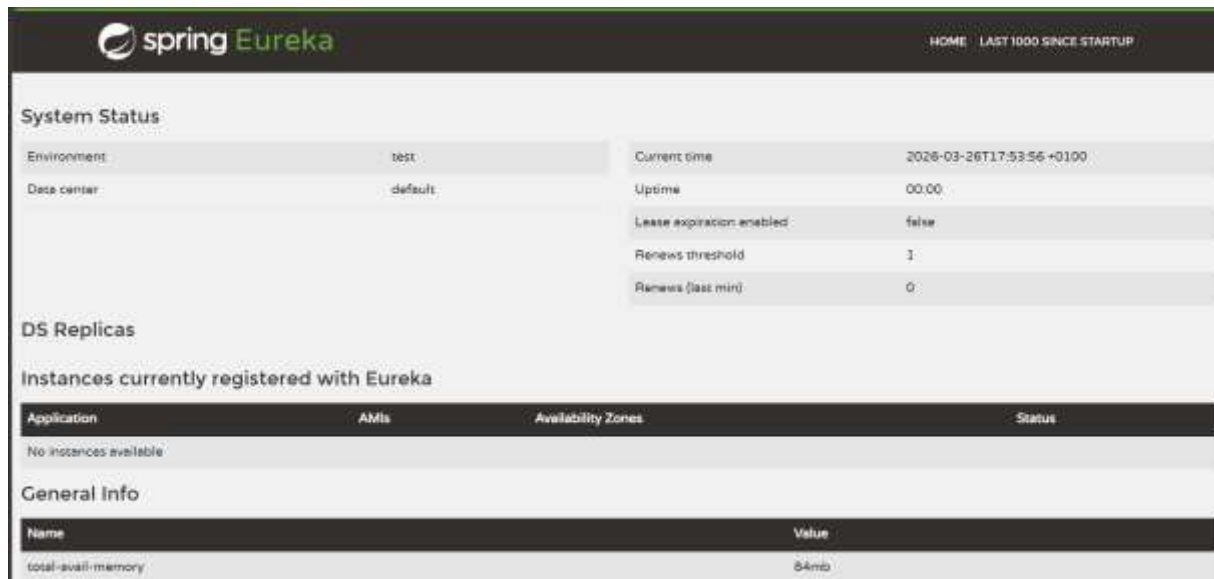
2-Classe principale

Dans cette partie, on active le serveur Eureka en ajoutant l'annotation **@EnableEurekaServer** dans la classe principale de l'application Spring Boot. Cela permet de transformer l'application en service de découverte capable d'enregistrer et de gérer les microservices. Une fois lancée, l'application démarre le serveur Eureka et rend accessible son tableau de bord pour superviser les services.

```
eureka-server > src > main > java > com > ecommerce > eureka_server > 📄 EurekaServerApplication.java
1  package com.ecommerce.eurekaserver;
2  import org.springframework.boot.SpringApplication;
3  import org.springframework.boot.autoconfigure.SpringBootApplication;
4  // Cette importation est indispensable pour activer le serveur Eureka
5  import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
6
7  @SpringBootApplication
8  @EnableEurekaServer // <-- C'est cette annotation qui active les fonctionnal
9  public class EurekaServerApplication {
10
11      public static void main(String[] args) {
12          // Cette ligne lance l'application Spring
13          SpringApplication.run(EurekaServerApplication.class, args);
14      }
15  }
```

3-Démarrage

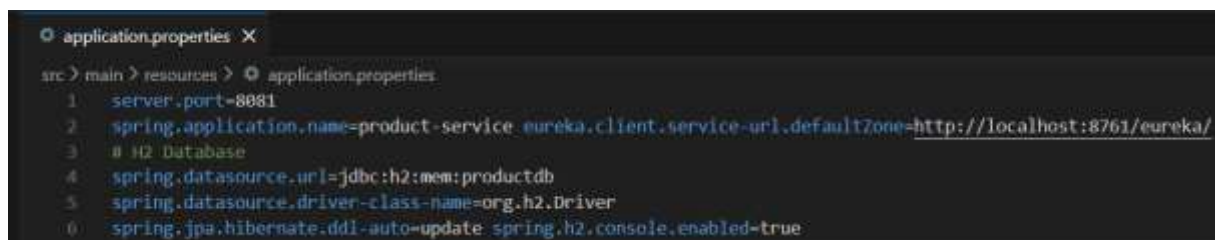
Une fois la configuration terminée, l'étape finale consiste à compiler et à lancer l'instance du serveur de découverte pour rendre le registre disponible aux futurs microservices.



L'affichage du tableau de bord Spring Eureka confirme que le moteur de découverte est actif et prêt à recevoir des requêtes d'enregistrement.

Étape 2 : Créer le service Product

1-Configuration



Dans cette partie, on met en place le Product Service, un microservice chargé de gérer le catalogue des produits. Le projet est créé via Spring Initializr avec les dépendances nécessaires pour exposer des API REST, gérer la base de données et s'enregistrer auprès d'Eureka. La configuration définit un port spécifique (8081), un nom de service pour l'identification dans Eureka, ainsi que les paramètres de la base de données H2 en mémoire. Cette étape permet d'avoir un service autonome capable de stocker et gérer les produits tout en étant intégré à l'architecture SOA.

2-Modèle de données

Créer la classe Product.java :

Ici on définit le modèle de données Product qui représente un produit dans l'application. Cette classe est annotée avec **@Entity** pour indiquer qu'elle correspond à une table dans la base de données, et contient les attributs essentiels comme l'identifiant, le nom, la

description, le prix et le stock. Les annotations de Lombok simplifient la création des getters, setters et constructeurs. Ce modèle permet de structurer et stocker les informations des produits de manière organisée dans la base de données.

```
src > main > java > com > ecommerce > product_service > model > J Product.java
1  package com.ecommerce.productservice.model;
2  import jakarta.persistence.*;
3  import lombok.*;
4  @Entity
5  @Data
6  @NoArgsConstructor
7  @AllArgsConstructor
8  public class Product {
9      @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
10     private String name;
11     private String description;
12     private Double price;
13     private Integer stock;
14 }
```

3-Repository

Dans cette partie, on crée le **repository** chargé d'accéder aux données des produits. L'interface **ProductRepository** étend **JpaRepository**, ce qui permet de bénéficier automatiquement des opérations CRUD (ajout, lecture, suppression, mise à jour) sans écrire de code supplémentaire. L'annotation **@Repository** indique que ce composant interagit avec la base de données. Cette couche facilite ainsi la gestion des données et la communication avec le modèle Product.

```
application.properties  J Product.java  J ProductRepository.java X
src > main > java > com > ecommerce > product_service > repository > J ProductRepository.java
1  package com.ecommerce.productservice.repository;
2
3  import com.ecommerce.productservice.model.Product;
4  import org.springframework.data.jpa.repository.JpaRepository;
5  import org.springframework.stereotype.Repository;
6
7  @Repository
8  public interface ProductRepository extends JpaRepository<Product, Long> {
9      // Grâce à JpaRepository, les méthodes findAll(), findById(), save(), etc.
10     // sont déjà prêtes à l'emploi sans écrire de code !
11 }
```

4-Service

Dans cette partie, on implémente la couche service qui contient la logique métier du microservice produit. La classe **ProductService** utilise le repository pour effectuer les opérations principales : récupérer tous les produits, chercher un produit par son ID, créer un nouveau produit et le supprimer. L'annotation **@Service** indique qu'il s'agit d'un composant métier, tandis que **@RequiredArgsConstructor** facilite l'injection du repository. Cette couche

permet de séparer la logique métier de l'accès aux données pour une meilleure organisation du code.

```
package com.ecommerce.productservice.service;
import com.ecommerce.productservice.model.Product;
import com.ecommerce.productservice.repository.ProductRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import java.util.List;
import java.util.Optional;
@Service
@RequiredArgsConstructor
public class ProductService {
    private final ProductRepository productRepository;
    public List<Product> getAllProducts() {
        return productRepository.findAll();
    }
    public Optional<Product> getProductById(Long id) {
        return productRepository.findById(id);
    }
    public Product createProduct(Product product) {
        return productRepository.save(product);
    }
    public void deleteProduct(Long id) {
        productRepository.deleteById(id);
    }
}
```

5-Contrôleur REST

Dans cette partie, on crée le contrôleur REST qui expose les API du service produit. La classe **ProductController** permet de gérer les requêtes HTTP pour afficher, ajouter et supprimer des produits en s'appuyant sur le **ProductService**. Elle définit les endpoints principaux de l'API et sert d'interface entre le client et la logique métier.

```
src > main > java > com > ecommerce > product_service > controller > ProductController.java
1 package com.ecommerce.productservice.controller;
2 import com.ecommerce.productservice.model.Product;
3 import com.ecommerce.productservice.service.ProductService;
4 import lombok.RequiredArgsConstructor;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.web.bind.annotation.*;
7 import java.util.List;
8 @RestController
9 @RequestMapping("/api/products")
10 @RequiredArgsConstructor
11 public class ProductController {
12     private final ProductService productService;
13     @GetMapping public List<Product> getAllProducts() { return productService.getAllProducts(); }
14     @GetMapping("/{id}") public ResponseEntity<Product> getProductById(@PathVariable long id) {
15         return productService.getProductById(id).map(ResponseEntity::ok).orElse(ResponseEntity.notFound().build());
16     }
17     @PostMapping public Product createProduct(@RequestBody Product product) { return productService.createProduct(product); }
18     @DeleteMapping("/{id}") public ResponseEntity<Void> deleteProduct(@PathVariable long id) {
19         productService.deleteProduct(id);
20         return ResponseEntity.noContent().build();
21     }
22 }
```

6-Activation Eureka Client

Dans cette partie, on active le client Eureka en ajoutant l'annotation **@EnableDiscoveryClient** dans la classe principale. Cela permet au service Product de s'enregistrer automatiquement auprès du serveur Eureka et d'être découvert par les autres microservices. Cette configuration facilite la communication et l'interaction dans l'architecture SOA.

```
src > main > java > com > ecommerce > product_service > ProductServiceApplication.java
1  package com.ecommerce.product_service;
2  import org.springframework.boot.SpringApplication;
3  import org.springframework.boot.autoconfigure.SpringBootApplication;
4  @SpringBootApplication
5  @EnableDiscoveryClient
6  public class ProductServiceApplication {
7      public static void main(String[] args) {
8          SpringApplication.run(ProductServiceApplication.class, args);
9      }
10 }
```

Étape 3 : Créer le service Order

1-Configuration

Dans cette partie, on crée le service **Order** en suivant le même processus que le service Product, mais avec des dépendances supplémentaires comme OpenFeign pour permettre la communication entre microservices. La configuration définit le port d'exécution (8082), le nom du service pour Eureka, ainsi que les paramètres de la base de données H2 en mémoire. Cette étape permet de mettre en place un microservice autonome chargé de la gestion des commandes, tout en étant intégré à l'architecture globale.

```
order-service > src > main > resources > application.properties
1  server.port=8082
2  spring.application.name=order-service eureka.client.service-url.defaultZone=
3  spring.datasource.url=jdbc:h2:mem:orderdb
4  spring.datasource.driver-class-name=org.h2.Driver
5  spring.jpa.hibernate.ddl-auto=update
6  spring.h2.console.enabled=true
7
```

2-Modèle de données

Dans cette partie, on définit le modèle de données **Order** qui représente une commande. La classe est annotée avec **@Entity** pour être persistée en base de données et contient les attributs nécessaires comme l'identifiant, l'ID du produit, le nom du produit, la quantité, le prix total, la date et le statut de la commande. Les annotations Lombok simplifient la gestion du code. Ce modèle permet de structurer et stocker les informations liées aux commandes.

```

order-service > src > main > java > com > ecommerce > order_service > model > J Order.java
1  package com.ecommerce.orderservice.model;
2  import jakarta.persistence.*;
3  import lombok.*;
4  import java.time.LocalDateTime;
5  @Entity
6  @Data
7  @NoArgsConstructor
8  @AllArgsConstructor
9  @Table(name = "orders")
10 public class Order {
11     @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
12     private Long productId;
13     private String productName;
14     private Integer quantity;
15     private Double totalPrice;
16     private LocalDateTime orderDate;
17     private String status; // PENDING, CONFIRMED, SHIPPED, DELIVERED
18 }

```

3-Client Feign pour Product Service

Ici, on met en place un client Feign pour permettre au service **Order** de communiquer avec le service Product. L'interface **ProductClient** utilise **@FeignClient** pour appeler directement les API du service product-service via son nom enregistré dans Eureka. Elle permet de récupérer les informations d'un produit à partir de son identifiant, facilitant ainsi la communication entre microservices de manière simple et transparente.

```

1  package com.ecommerce.orderservice.client;
2  import com.ecommerce.orderservice.dto.ProductDTO;
3  import org.springframework.cloud.openfeign.FeignClient;
4  import org.springframework.web.bind.annotation.GetMapping;
5  import org.springframework.web.bind.annotation.PathVariable;
6  @FeignClient(name = "product-service")
7  public interface ProductClient {
8      @GetMapping("/api/products/{id}") ProductDTO getProductById(@PathVariable
9  }

```

4-DTO pour Product

Dans cette partie, on crée un **DTO (Data Transfer Object)** pour représenter les données du produit lors des échanges entre microservices. La classe **ProductDTO** permet de transférer uniquement les informations nécessaires sans exposer directement l'entité de la base de données. Les annotations Lombok simplifient la création des getters, setters et constructeurs. Ce DTO facilite ainsi la communication et le découplage entre les services.


```

order-service > src > main > java > com > ecommerce > order_service > dto > J ProductDTO.java
1  import lombok.*;
2  @Data
3  @NoArgsConstructor
4  @AllArgsConstructor
5  public class ProductDTO {
6      private Long id;
7      private String name;
8      private String description;
9      private Double price;
10     private Integer stock;
11 }

```

5-Repository et Service

OrderRepository.java :

```

1  package com.ecommerce.orderservice.repository;
2  import com.ecommerce.orderservice.model.Order;
3  import org.springframework.data.jpa.repository.JpaRepository;
4  import org.springframework.stereotype.Repository;
5  @Repository
6  public interface OrderRepository extends JpaRepository < Order, Long > {}

```

OrderRepository hérite de JpaRepository, ce qui permet de gérer facilement les opérations sur la base de données (ajout, suppression, recherche...) pour l'entité Order sans écrire de code SQL.

OrderService.java :

```

order-service > src > main > java > com > ecommerce > order_service > service > J OrderService.java
1  package com.ecommerce.orderservice.service;
2  import com.ecommerce.orderservice.client.ProductClient;
3  import com.ecommerce.orderservice.dto.ProductDTO;
4  import com.ecommerce.orderservice.model.Order;
5  import com.ecommerce.orderservice.repository.OrderRepository;
6  import lombok.RequiredArgsConstructor;
7  import org.springframework.stereotype.Service;
8  import java.time.LocalDateTime;
9  import java.util.List;
10 @Service
11 @RequiredArgsConstructor
12 public class OrderService {
13     private final OrderRepository orderRepository;
14     private final ProductClient productClient;
15     public List < Order > getAllOrders() {
16         return orderRepository.findAll();
17     }
18     public Order createOrder(Long productId, Integer quantity) {
19         ProductDTO product = productClient.getProductById(productId);
20         Order order = new Order();
21         order.setProductId(productId);
22         order.setProductName(product.getName());
23         order.setQuantity(quantity);
24         order.setTotalPrice(product.getPrice() * quantity);
25         order.setOrderDate(LocalDateTime.now());
26         order.setStatus("PENDING");
27         return orderRepository.save(order);
28     }
29 }

```

6-Contrôleur

Cette étape consiste à créer le contrôleur REST du service Order . OrderController expose les endpoints API pour interagir avec les commandes :

- GET /api/orders : récupère la liste de toutes les commandes.
- POST /api/orders : crée une nouvelle commande en appelant le service métier (OrderService).

```
order-service > src > main > java > com > ecommerce > order_service > controller > J OrderController.java
1  package com.ecommerce.orderservice.controller;
2  import com.ecommerce.orderservice.model.Order;
3  import com.ecommerce.orderservice.service.OrderService;
4  import lombok.RequiredArgsConstructor;
5  import org.springframework.web.bind.annotation.*;
6  import java.util.List;
7  @RestController
8  @RequestMapping("/api/orders")
9  @RequiredArgsConstructor
10 public class OrderController {
11     private final OrderService orderService;
12     @GetMapping public List < Order > getAllOrders() {
13         return orderService.getAllOrders();
14     }
15     @PostMapping public Order createOrder(@RequestParam Long productId, @RequestParam Integer quantity) {
16         return orderService.createOrder(productId, quantity);
17     }
18 }
```

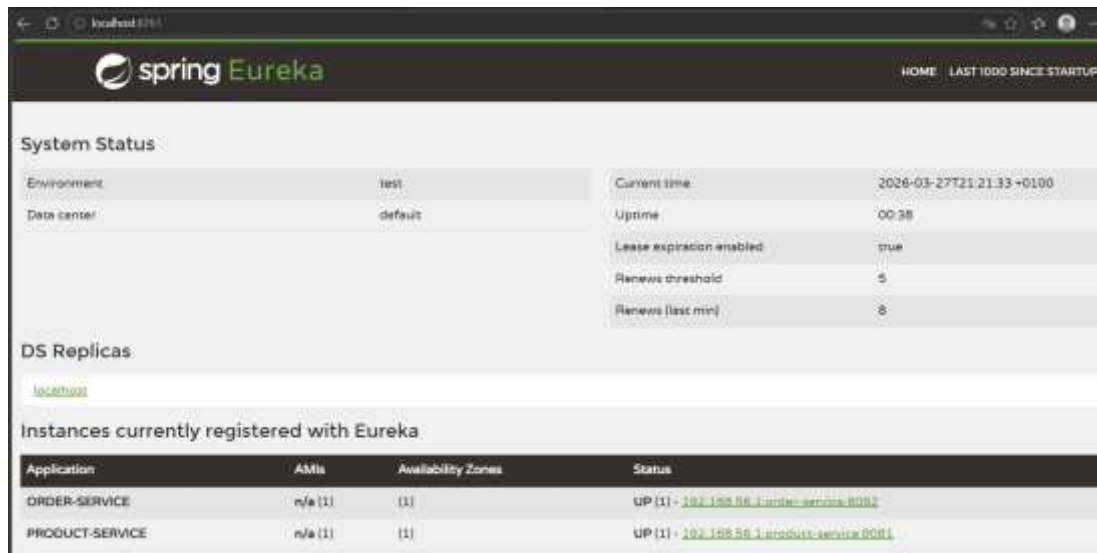
7-Activation des clients Feign

```
order-service > src > main > java > com > ecommerce > order_service > J OrderServiceApplication.java
1  package com.ecommerce.order_service;
2  import org.springframework.boot.SpringApplication;
3  import org.springframework.boot.autoconfigure.SpringBootApplication;
4  @SpringBootApplication
5  @EnableDiscoveryClient
6  @EnableFeignClients
7  public class OrderServiceApplication {
8
9      public static void main(String[] args) {
10         SpringApplication.run(OrderServiceApplication.class, args);
11     }
12
13 }
```

L'activation de Feign et du Discovery Client marque la fin de la phase de développement du order-service. Le microservice est maintenant prêt à consommer les données du product-service de manière synchrone, illustrant le succès de la mise en réseau des composants de l'application e-commerce.

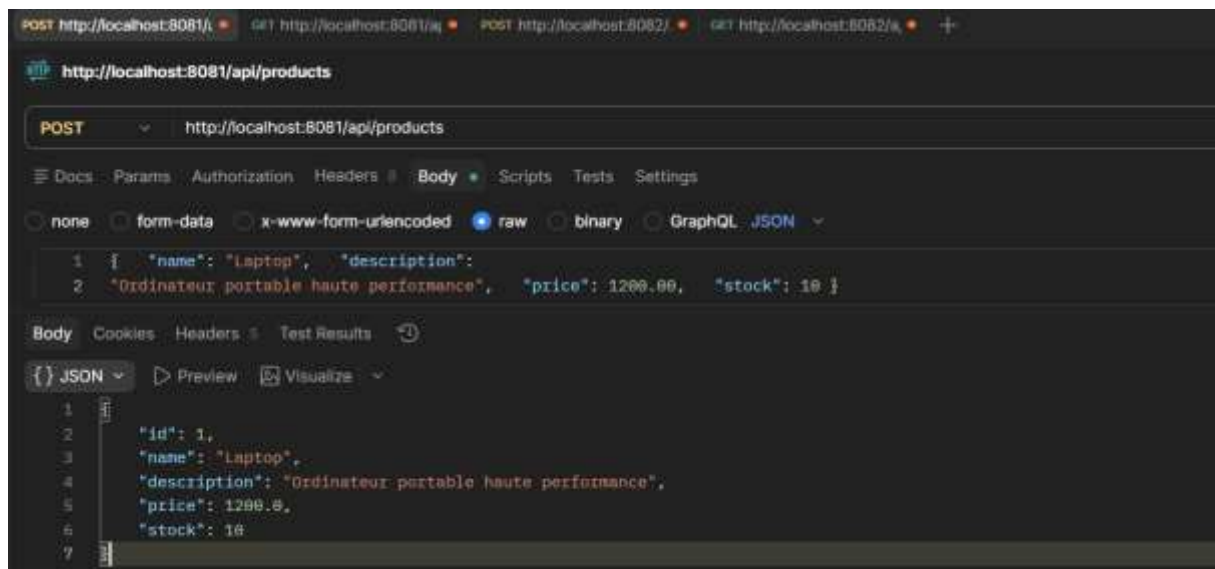
Étape 4 : Tester l'application

Les tests effectués avec Postman démontrent la robustesse de l'architecture. Le découplage est total : chaque service gère ses données, mais collabore de manière transparente grâce à OpenFeign. L'application e-commerce est désormais fonctionnelle, scalable et prête à recevoir de nouveaux modules.

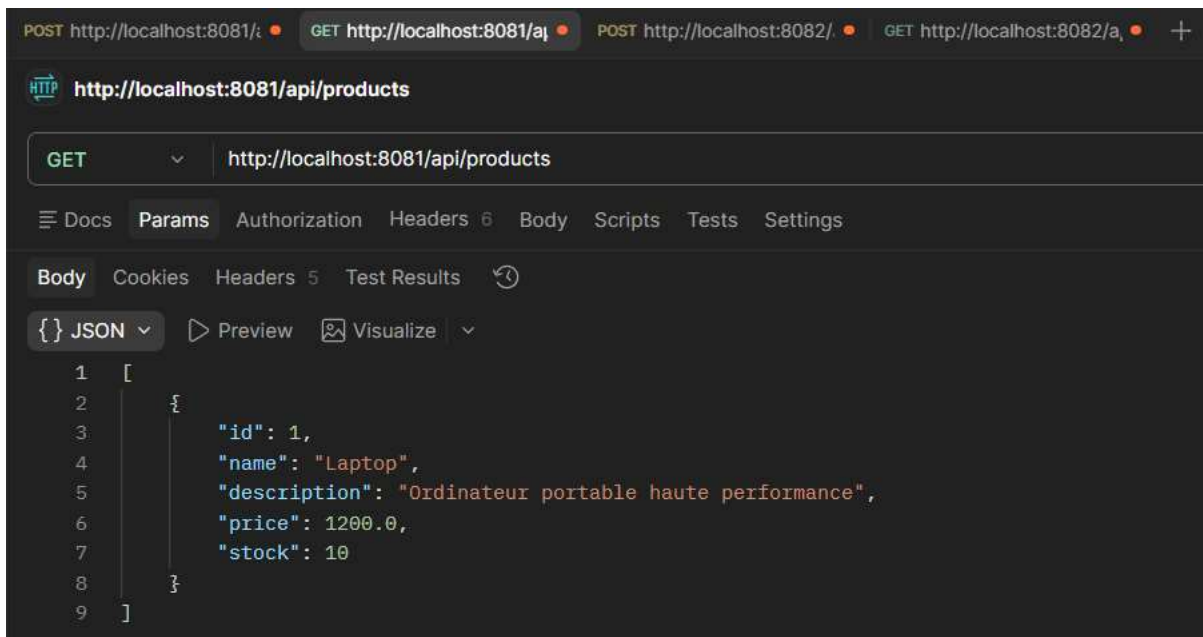


Tests avec Postman

Test 1 : Créer un produit



Test 2 : Lister les produits



POST http://localhost:8081/ GET http://localhost:8081/api/ POST http://localhost:8082/ GET http://localhost:8082/api/ +

http://localhost:8081/api/products

GET http://localhost:8081/api/products

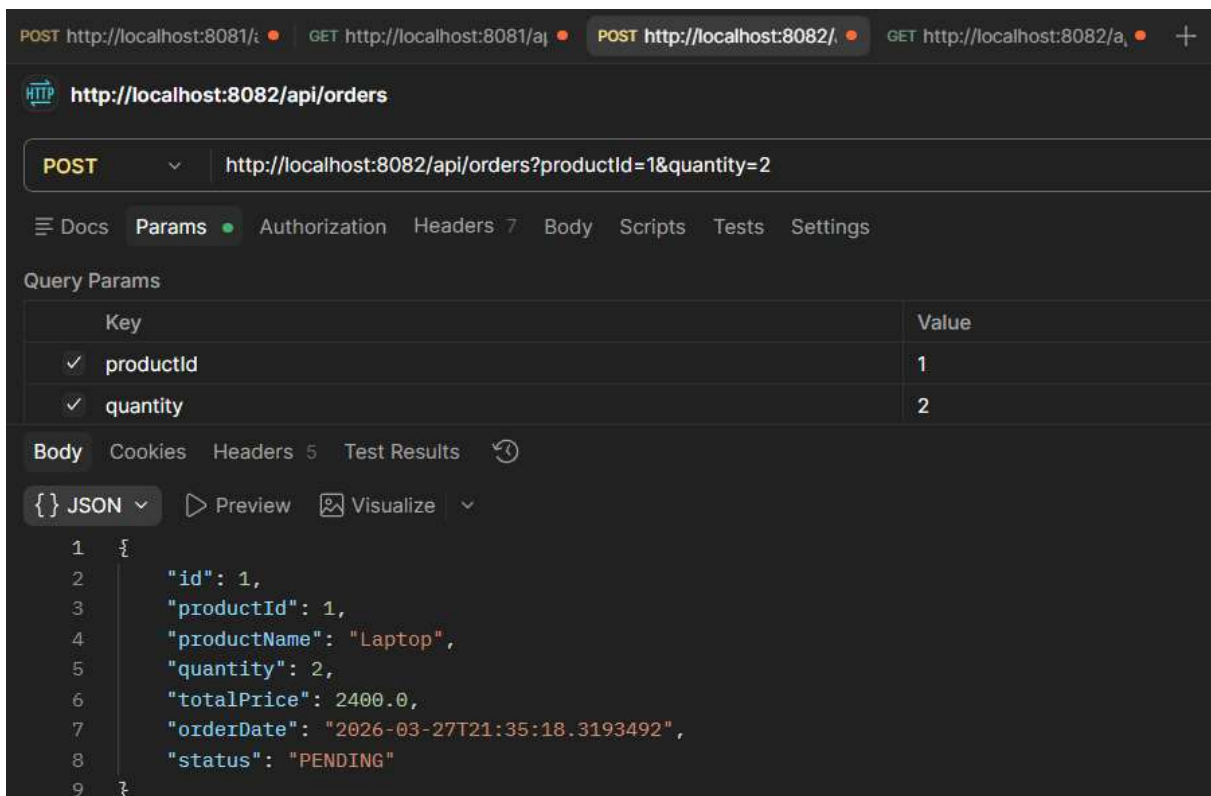
Docs Params Authorization Headers 6 Body Scripts Tests Settings

Body Cookies Headers 5 Test Results ↺

{ } JSON Preview Visualize

```
1 [
2   {
3     "id": 1,
4     "name": "Laptop",
5     "description": "Ordinateur portable haute performance",
6     "price": 1200.0,
7     "stock": 10
8   }
9 ]
```

Test 3 : Créer une commande



POST http://localhost:8081/ GET http://localhost:8081/api/ POST http://localhost:8082/ GET http://localhost:8082/api/ +

http://localhost:8082/api/orders

POST http://localhost:8082/api/orders?productId=1&quantity=2

Docs Params Authorization Headers 7 Body Scripts Tests Settings

Query Params

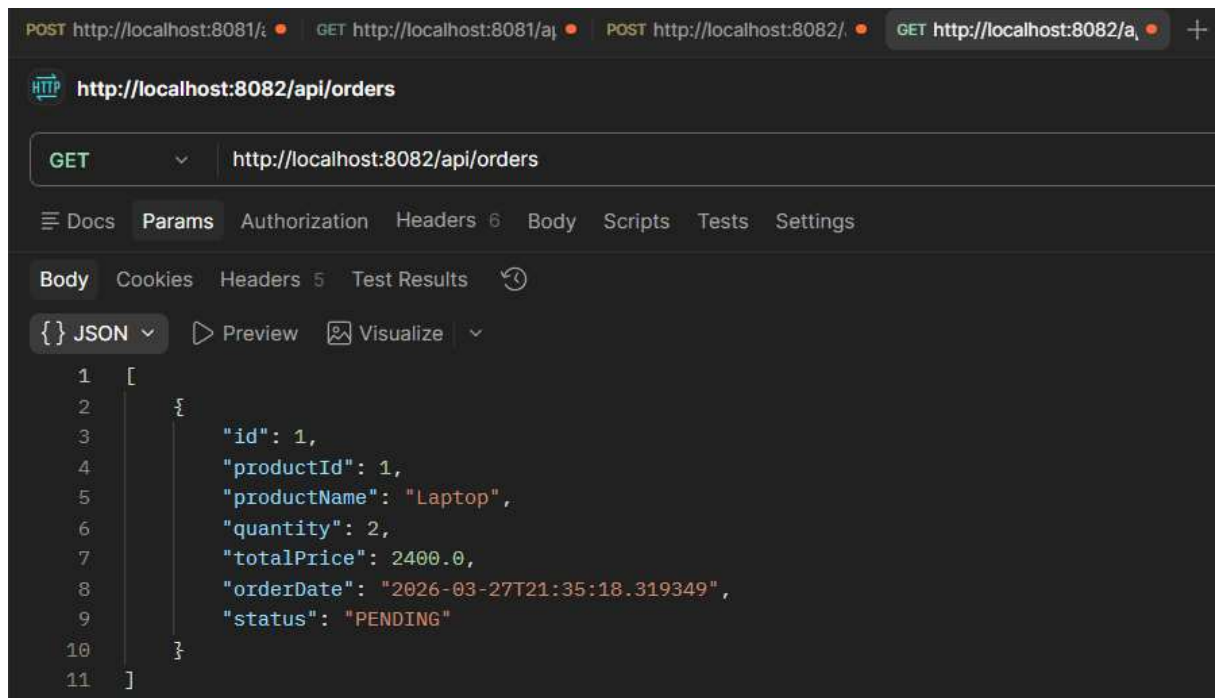
Key	Value
✓ productId	1
✓ quantity	2

Body Cookies Headers 5 Test Results ↺

{ } JSON Preview Visualize

```
1 {
2   "id": 1,
3   "productId": 1,
4   "productName": "Laptop",
5   "quantity": 2,
6   "totalPrice": 2400.0,
7   "orderDate": "2026-03-27T21:35:18.3193492",
8   "status": "PENDING"
9 }
```

Test 4 : Lister les commandes

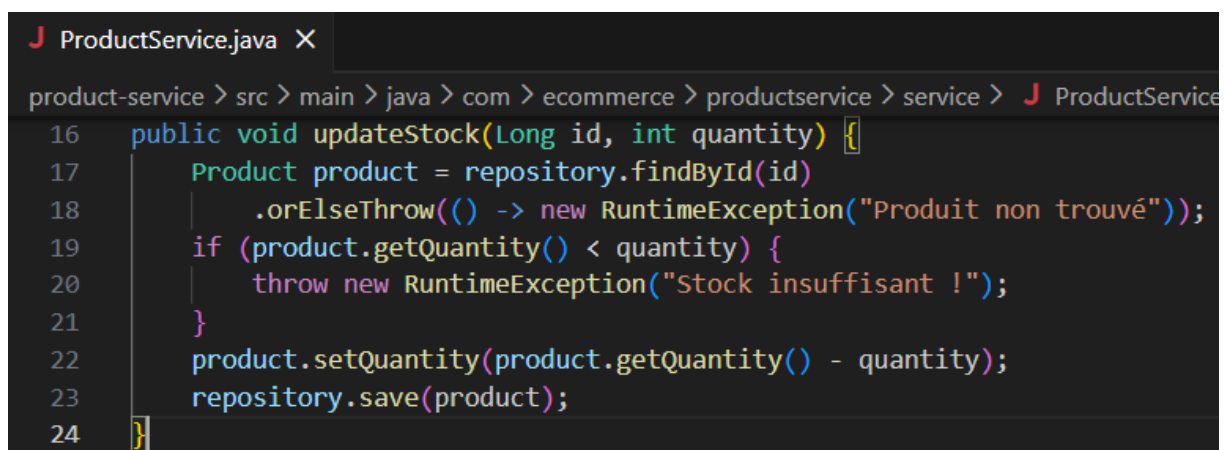


Exercices pratiques

Exercice 1 : Ajouter une fonctionnalité

Dans le Product Service (Le Fournisseur)

Fichier ProductService.java : Ajout une méthode pour modifier le stock.



Fichier ProductController.java : Création de l'endpoint PUT

Dans le Order Service (Le Consommateur)

Fichier ProductClient.java (Feign) : Ajout de la signature de la méthode PUT.

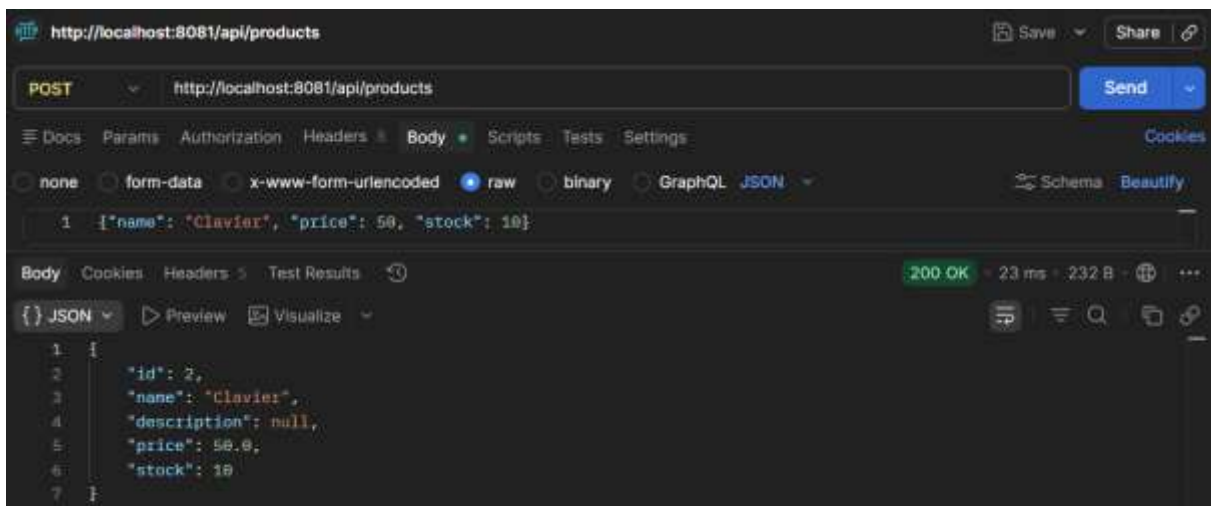
```
ProductClient.java •
order-service > order-service > src > main > java > com > ecommerce > orderservice > client > ProductClient.java
10 @PutMapping("/api/products/{id}/reduce-stock")
11 void reduceStock(@PathVariable("id") Long id, @RequestParam("quantity") int quantity);
```

Fichier OrderService.java : Modification de la méthode de création de commande.

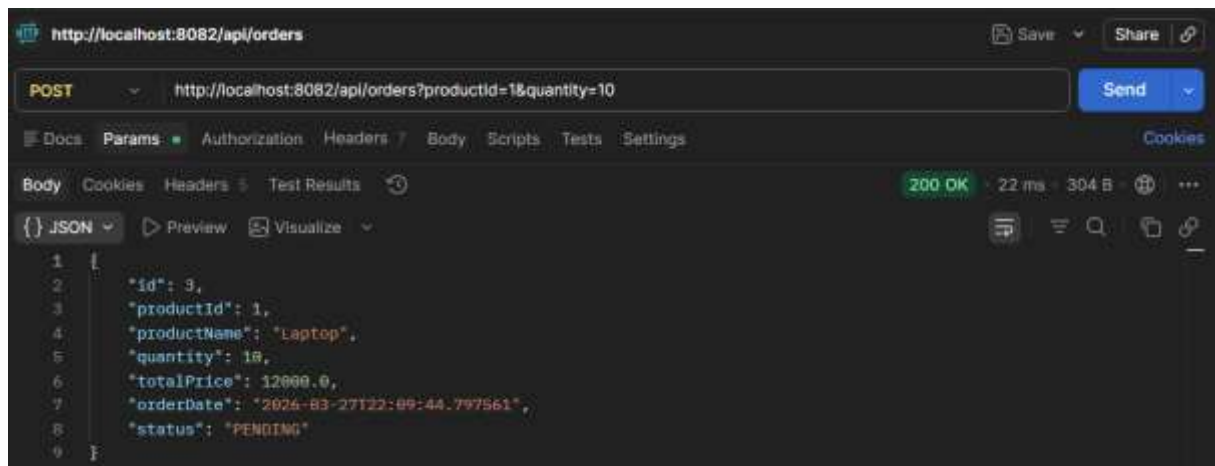
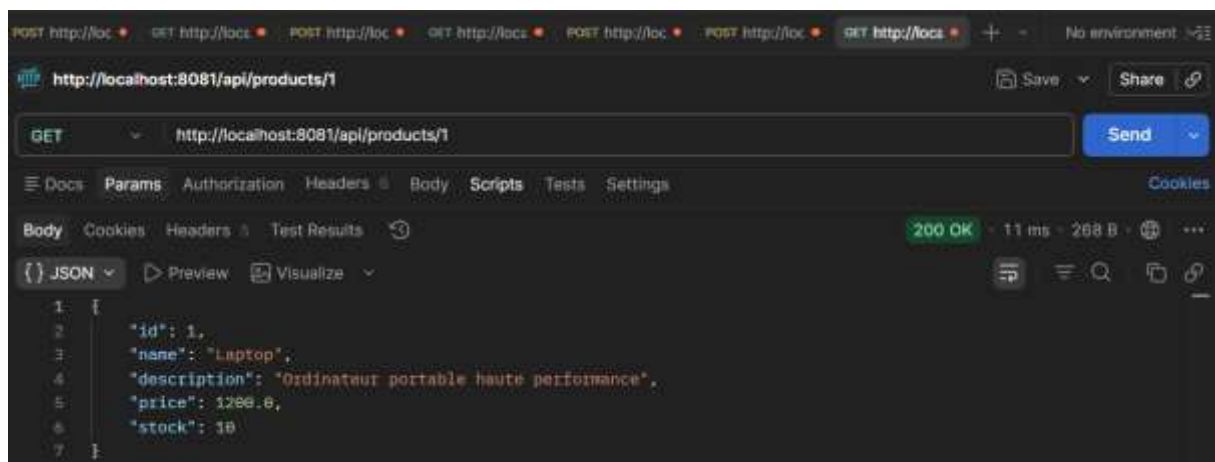
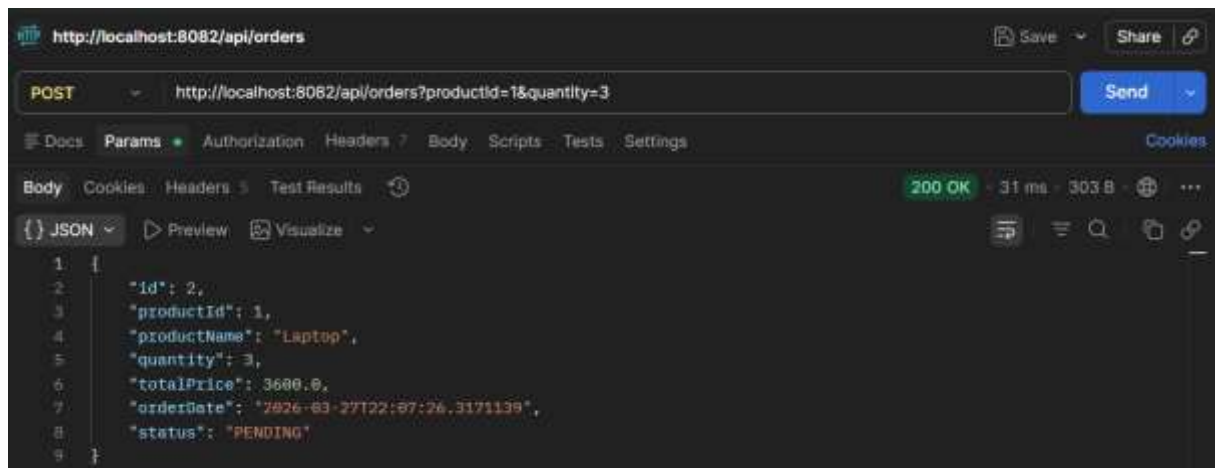
```
ProductClient.java • OrderService.java X
order-service > order-service > src > main > java > com > ecommerce > orderservice > service > OrderService.java
12 public class OrderService {
13
14     public Order createOrder(Long productId, Integer quantity) {
15         // 1. Récupérer les infos du produit (tu l'as déjà fait)
16         ProductDTO product = productClient.getProductById(productId);
17
18         // 2. AJOUT : Appeler le client Feign pour réduire le stock
19         // Si le stock est insuffisant, une exception sera levée ici et le code s'arrêtera
20         productClient.reduceStock(productId, quantity);
21
22         // 3. Créer la commande (ton code existant)
23         Order order = new Order();
24         order.setProductId(productId);
25         order.setProductName(product.getName());
26         order.setQuantity(quantity);
27         order.setTotalPrice(product.getPrice() * quantity);
28         order.setOrderDate(LocalDateTime.now());
29         order.setStatus("CREATED"); // On peut passer de PENDING à CREATED car le stock est va
30
31         return orderRepository.save(order);
32     }
33 }
```

Teste

1-Initialiser un produit avec du stock



2-Test du scénario nominal (Succès)



Le succès de ces tests démontre le couplage fort entre la validation du stock et la création de la commande. L'utilisation d'OpenFeign permet ici de simuler une transaction distribuée simple : l'échec de la mise à jour du stock entraîne l'échec de l'ensemble du processus de commande. C'est une illustration concrète de la gestion de l'intégrité des données dans une architecture microservices sans utiliser de gestionnaire de transactions complexes (comme SAGA).

Exercice 2 : Créer une API Gateway

1-Dépendances (pom.xml)

À l'intérieur de ce nouveau dossier gateway-service, crée un fichier nommé pom.xml

```
gateway-service > pom.xml
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4    <modelVersion>4.0.0</modelVersion>
5    <parent>
6      <groupId>org.springframework.boot</groupId>
7      <artifactId>spring-boot-starter-parent</artifactId>
8      <version>3.2.5</version>
9      <relativePath></relativePath>
10   </parent>
11   <groupId>com.ecommerce</groupId>
12   <artifactId>gateway-service</artifactId>
13   <version>0.0.1-SNAPSHOT</version>
14   <name>gateway-service</name>
15   <properties>
16     <java.version>21</java.version>
17     <spring.cloud.version>2023.0.1</spring.cloud.version>
18   </properties>
19   <dependencies>
20     <dependency>
21       <groupId>org.springframework.cloud</groupId>
22       <artifactId>spring-cloud-starter-gateway</artifactId>
23     </dependency>
24     <dependency>
25       <groupId>org.springframework.cloud</groupId>
26       <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
27     </dependency>
28     <dependency>
29       <groupId>org.springframework.boot</groupId>
30       <artifactId>spring-boot-starter-test</artifactId>
31       <scope>test</scope>
32     </dependency>
33   </dependencies>
34   <dependencyManagement>
35     <dependencies>
36       <dependency>
37         <groupId>org.springframework.cloud</groupId>
38         <artifactId>spring-cloud-dependencies</artifactId>
39         <version>${spring.cloud.version}</version>
40         <type>pom</type>
41         <scope>import</scope>
42       </dependency>
43     </dependencies>
44   </dependencyManagement>
45   <build>
46     <plugins>
47       <plugin>
48         <groupId>org.springframework.boot</groupId>
49         <artifactId>spring-boot-maven-plugin</artifactId>
50       </plugin>
51     </plugins>
52   </build>
53 </project>
```

Création de la classe principale

Crée ensuite le chemin de dossiers : src/main/java/com/ecommerce/gatewayservice/ et crée-y le fichier GatewayServiceApplication.java


```
GatewayServiceApplication.java X
gateway-service > src > main > java > com > ecommerce > gateway-service > GatewayServiceApplication.java
1 package com.ecommerce.gateway-service;
2 import org.springframework.boot.SpringApplication;
3 import org.springframework.boot.autoconfigure.SpringBootApplication;
4 import org.springframework.cloud.client.discovery.EnableDiscoveryClient;
5 @SpringBootApplication
6 @EnableDiscoveryClient
7 public class GatewayServiceApplication {
8     public static void main(String[] args) {
9         SpringApplication.run(GatewayServiceApplication.class, args);
10    }
11 }
```

2-Configuration (application.properties)

dans le dossier gateway-service/src/main/resources/ et crée (ou modifie) le fichier application.properties

```
GatewayServiceApplication.java application.properties X
gateway-service > src > main > resources > application.properties
1 # Port d'entrée unique pour le client (Postman/Front-end)
2 server.port=8080
3 spring.application.name=gateway-service
4 # Connexion à l'annuaire Eureka
5 eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
6 # --- CONFIGURATION DES ROUTES ---
7 # Route 1 : Redirection vers le Product Service
8 spring.cloud.gateway.routes[0].id=product-service-route
9 spring.cloud.gateway.routes[0].uri=lb://PRODUCT-SERVICE
10 spring.cloud.gateway.routes[0].predicates[0]=Path=/api/products/**
11 # Route 2 : Redirection vers le Order Service
12 spring.cloud.gateway.routes[1].id=order-service-route
13 spring.cloud.gateway.routes[1].uri=lb://ORDER-SERVICE
14 spring.cloud.gateway.routes[1].predicates[0]=Path=/api/orders/**
```

3-OrderController.java

```
application.properties  Order.java  OrderController.java
order-service > order-service > src > main > java > com > ecommerce > orderservice > controller > OrderController.java

4  import com.ecommerce.orderservice.service.OrderService;
5  import lombok.Data;
6  import lombok.RequiredArgsConstructor;
7  import org.springframework.web.bind.annotation.*;
8  import java.util.List;
9  @RestController
10 @RequestMapping("/api/orders")
11 @RequiredArgsConstructor
12 public class OrderController {
13     private final OrderService orderService;
14     @GetMapping
15     public List<Order> getAllOrders() {
16         return orderService.getAllOrders();
17     }
18     // CORRECTION : On utilise @RequestBody avec un objet DTO
19     @PostMapping
20     public Order createOrder(@RequestBody OrderRequest request) {
21         return orderService.createOrder(request.getProductId(), request.getQuantity());
22     }
23     // Cette classe permet de mapper le JSON de Postman
24     @Data
25     public static class OrderRequest {
26         private Long productId;
27         private Integer quantity;
28     }
29 }
```

Teste :

➔ Créer le produit :

```
http://localhost:8080/api/products

POST http://localhost:8080/api/products

Docs Params Authorization Headers Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

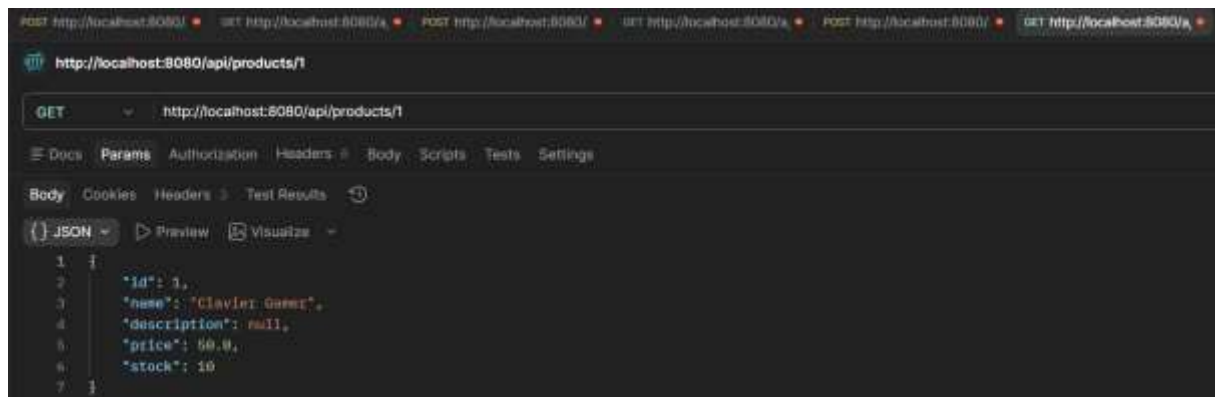
1 {
2   "name": "Clavier Gamer",
3   "price": 50.0,
4   "stock": 10
5 }

Body Cookies Headers Test Results

{} JSON Preview Visualize

1 {
2   "id": 1,
3   "name": "Clavier Gamer",
4   "description": null,
5   "price": 50.0,
6   "stock": 10
7 }
```

➔ Vérifier que le produit existe :



➔ Passer la commande :

